ntic master

UNIVERSIDAD COMPLUTENSE MADRID



DOCUMENTACIÓN

MODULE / COURSE
Bases de Datos SQL

TOPIC/UNIT
BLOQUE 2: Lenguaje SQL

PROFESSOR:
Juan Fernando Sánchez Martínez

Unit 4. Lenguaje SQL

a. **DDL (Data Definition Language)**

   i. ALTER - Modificación de Estructuras

   ii. DROP - Eliminación de Objetos

b. **DML (Data Manipulation Language)**

   i. **Consultas Básicas**

      1. UPDATE/DELETE - Modificación
      2. SELECT - Selección de Datos
      3. DISTINCT/AS - Alias y Únicos
      4. WHERE - Filtrado
      5. ORDER BY - Ordenación
      6. Funciones
      7. GROUP BY - Agrupación

   ii. **Combinación de Tablas**

      1. JOIN - Relaciones entre Tablas
         a. INNER JOIN
         b. LEFT JOIN
         c. RIGHT JOIN
         d. FULL OUTER JOIN

   iii. **Consultas Avanzadas**

      1. SUBQUERIES - Subconsultas
      2. CASE WHEN - Condicionales

3. VIEWS - Vistas
4. LIKE - Patrones
5. TOP - Límites
6. HAVING - Filtrado de Grupos
7. WINDOW FUNCTIONS - Funciones de Ventana
8. CTEs - Common Table Expressions
9. SET OPERATORS - Operadores de Conjunto

1. INTRODUCCIÓN

Las instrucciones SQL se pueden dividir en dos categorías dependiendo de la función que realizan. Estas categorías son el Lenguaje de Definición de Datos (DDL) y el Lenguaje de Manipulación de Datos (DML).

1. **Lenguaje de Definición de Datos** (DDL, Data Definition Language): se utiliza para crear, modificar o eliminar objetos de una base de datos, como tablas, esquemas, vistas o procedimientos almacenados. Este tipo de instrucciones suelen empezar por las palabras clave CREATE, ALTER y DROP. Por ejemplo, se puede usar la instrucción CREATE TABLE para crear una tabla, ALTER TABLE para modificarla y DROP TABLE para eliminarla. Un SGBD es una herramienta que permite trabajar con el Lenguaje de Definición de Datos de manera visual y estructurada. Con esta herramienta se pueden crear y administrar bases de datos, ejecutar consultas SQL y realizar tareas de administración del servidor.
2. **Lenguaje de Manipulación de Datos** (DML, Data manipulation Language**):** se utiliza para manipular los datos de una base de datos, es decir, recuperar, añadir, modificar o borrar datos almacenados en ella. Este tipo de instrucciones suelen empezar por las palabras clave SELECT, INSERT, UPDATE y DELETE. Por ejemplo, se puede usar la instrucción SELECT para recuperar datos de varias tablas y la instrucción INSERT para insertar datos en una tabla. Un SGBD también permite trabajar con el Lenguaje de Manipulación de Datos de manera visual y estructurada. Con esta herramienta se pueden ejecutar consultas SQL y manipular los datos de una base de datos de manera eficiente.

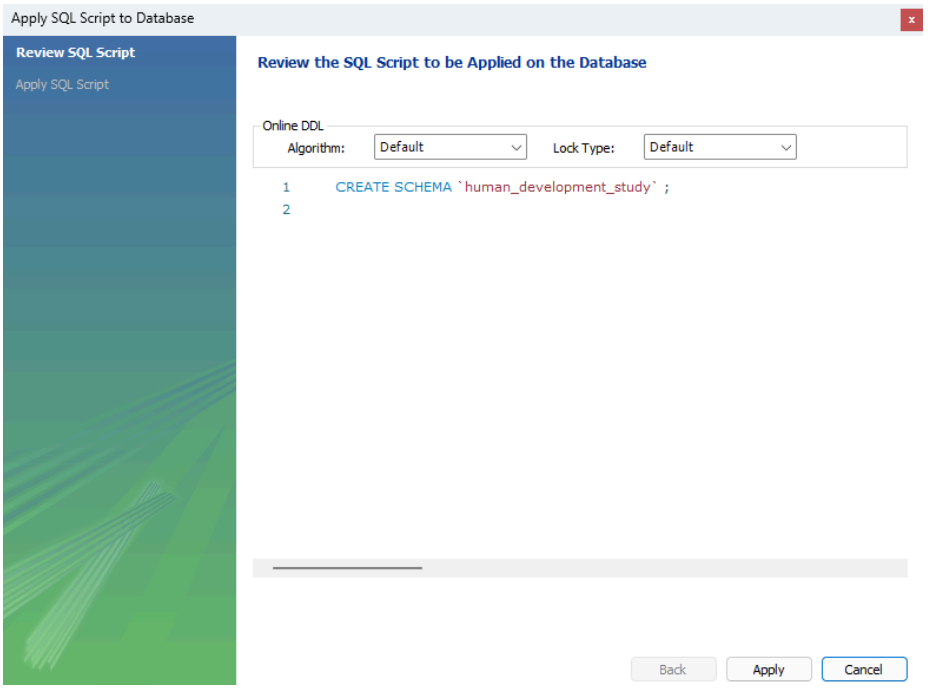
2. LENGUAJE DE DEFINICIÓN DE DATOS (DDL)

El **Lenguaje de Definición de Datos (DDL)** es una parte fundamental de SQL que se utiliza para definir la estructura de una base de datos. En SQL, el DDL se utiliza para crear, modificar o eliminar objetos de una base de datos, como tablas, esquemas, vistas o procedimientos almacenados. Las sentencias DDL más comunes son CREATE, ALTER y DROP.

La sentencia CREATE se utiliza para crear nuevos objetos en la base de datos, como tablas, vistas o procedimientos almacenados. Por ejemplo, se puede utilizar la sentencia CREATE TABLE para crear una nueva tabla en la base de datos. La sentencia ALTER se utiliza para modificar objetos existentes en la base de datos, como agregar o eliminar columnas de una tabla. La sentencia DROP se utiliza para eliminar **objetos** de la base de datos, como tablas o vistas.

CREATE

Para a crear nuestra Base de Datos mediante el uso de la sentencia CREATE SCHEMA human_development_study:



A continuación crearemos una tabla mediante DDL donde después cargaremos los datos. La sentencia para crear una tabla es CREATE TABLE. La sintaxis tendrá la siguiente estructura:

CREATE TABLE nombre_schema.nombre tabla (

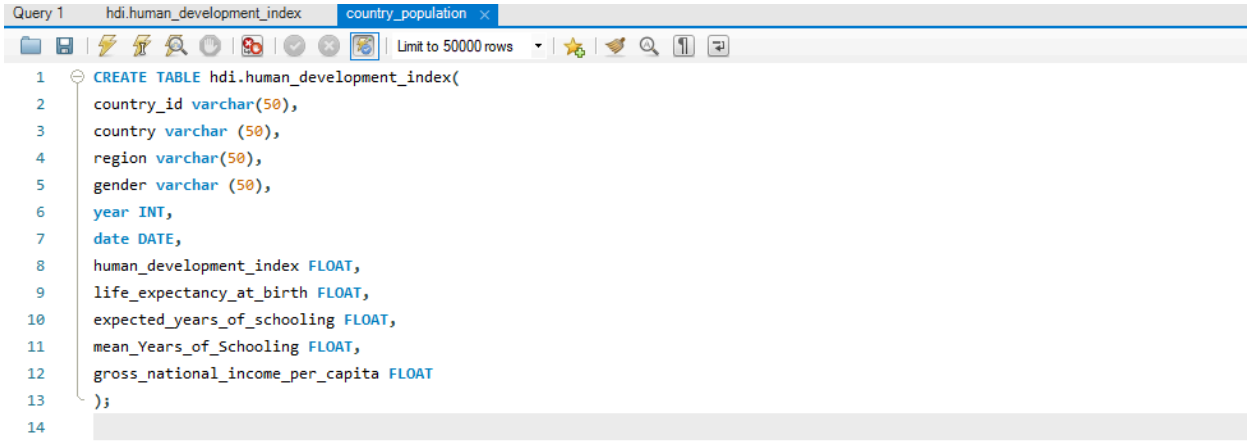
columna1 TIPO DE DATO,

columna1 TIPO DE DATO,

columna1 TIPO DE DATO,

…)

La tabla de ejemplo que vamos a crear en Un SGBD es la tabla *human_development_index* donde indicaremos el tipo de dato que corresponde a cada columna.



3. LENGUAJE DE MANIPULACIÓN DE DATOS (DML)

El **Lenguaje de Manipulación de Datos (DML)** es un subconjunto de SQL que se utiliza para gestionar y manipular datos en una base de datos relacional. En SQL, el

DML se utiliza para recuperar, añadir, modificar o borrar datos almacenados en la base de datos. Las sentencias DML más comunes son SELECT, INSERT, UPDATE y DELETE.

La sentencia SELECT se utiliza para recuperar datos de una o varias tablas de la base de datos. Por ejemplo, se puede utilizar la sentencia SELECT para recuperar los nombres de todos los clientes que han realizado una compra en una tienda en línea. La sentencia INSERT se utiliza para añadir nuevos datos a una tabla de la base de datos. Por ejemplo, se puede utilizar la sentencia INSERT para añadir un nuevo cliente a la tabla de clientes. La sentencia UPDATE se utiliza para modificar datos existentes en una tabla de la base de datos. Por ejemplo, se puede utilizar la sentencia UPDATE para cambiar la dirección de un cliente en la tabla de clientes. La sentencia DELETE se utiliza para eliminar datos de una tabla de la base de datos. Por ejemplo, se puede utilizar la sentencia DELETE para eliminar un cliente de la tabla de clientes.

Un SGBD es una herramienta que permite trabajar con el DML de manera visual y estructurada. Con esta herramienta se pueden ejecutar consultas SQL y manipular los datos de una base de datos de manera eficiente. Además, Un SGBD proporciona una interfaz gráfica para trabajar con las bases de datos de manera estructurada.

a. Consultas Básicas

UPDATE Y DELETE

Ya hemos aprendido cómo crear tablas e insertar datos. Ahora es momento de dominar dos operaciones fundamentales para el mantenimiento de datos: la modificación y eliminación de registros existentes. Las instrucciones UPDATE y DELETE son herramientas esenciales que permiten mantener la integridad y actualidad de la información almacenada en nuestras bases de datos.

La instrucción UPDATE permite modificar los valores de uno o más campos en registros existentes de una tabla. Esta operación es crucial cuando necesitamos corregir información errónea, actualizar datos que han cambiado con el tiempo, o realizar modificaciones masivas en múltiples registros simultáneamente.

La sintaxis básica de UPDATE incluye varias cláusulas que nos permiten especificar exactamente qué registros modificar y cómo hacerlo de manera precisa y segura.

UPDATE nombre_tabla

SET columna1 = nuevo_valor1,

columna2 = nuevo_valor2

WHERE condición;

Consideraciones Importantes:

- La cláusula WHERE es fundamental para evitar modificar registros no deseados
- Sin WHERE, se actualizarán TODOS los registros de la tabla
- Es recomendable hacer un SELECT previo para verificar qué registros se verán afectados

DELETE - Eliminación de Datos

La instrucción DELETE permite eliminar registros completos de una tabla cuando ya no son necesarios o contienen información incorrecta que no puede ser corregida. A diferencia de UPDATE, que modifica campos específicos, DELETE remueve registros enteros de la base de datos.

Esta operación debe utilizarse con extrema precaución, ya que una vez eliminados los datos, su recuperación puede ser compleja o imposible sin respaldos adecuados.

Sintaxis Básica:

DELETE FROM nombre_tabla

WHERE condición;

Consideraciones Importantes:

- La cláusula WHERE es CRÍTICA - sin ella se eliminarán TODOS los registros

- DELETE elimina registros completos, no campos individuales
- Siempre verificar la condición con un SELECT antes de ejecutar DELETE
- Considerar hacer respaldos antes de operaciones DELETE masivas

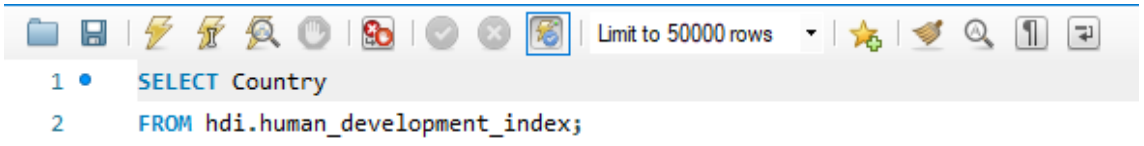
Tanto UPDATE como DELETE son operaciones irreversibles que requieren cuidado especial. Es fundamental comprender su impacto antes de ejecutarlas, ya que son las herramientas que nos permiten mantener nuestros datos actualizados y limpios, pero también las que pueden causar pérdida de información si se utilizan incorrectamente.

SELECT Y FROM

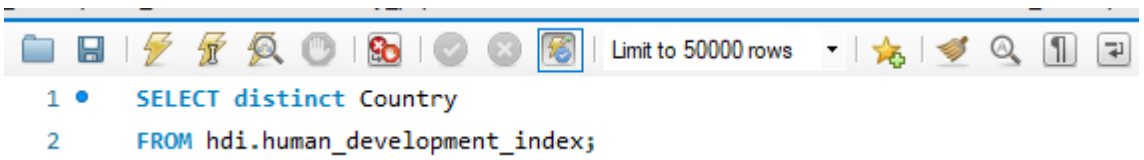
Ya hemos visto cómo crear tablas y poblarlas de datos. Ahora veremos cómo recuperar información específica de la base de datos. La instrucción SELECT permite formar consultas que devuelven los datos que se desean recuperar. Es una de las instrucciones más comunes que utilizan los desarrolladores de SQL. La sintaxis básica de la instrucción SELECT puede dividirse en varias cláusulas específicas que ayudan a refinar la consulta para que devuelva los datos requeridos.

DISTINCT y ALL

La cláusula SELECT incluye las palabras clave DISTINCT y ALL.

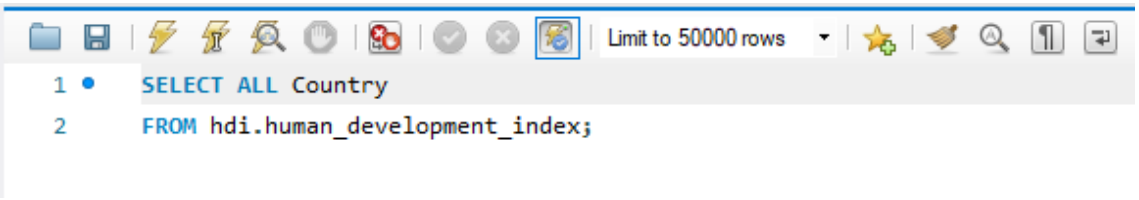


DISTINCT se utiliza para eliminar filas duplicadas de los resultados de una consulta. ALL se utiliza para devolver todas las filas de una consulta. Si no se especifica ninguna de las dos se toma por defecto la palabra clave ALL.



ALIAS

Los alias en SQL son una forma de asignar un nombre temporal a una tabla o a una columna en una consulta. Esto puede ser útil para hacer que las consultas sean más legibles y para trabajar con tablas y columnas que tienen nombres largos o complicados.

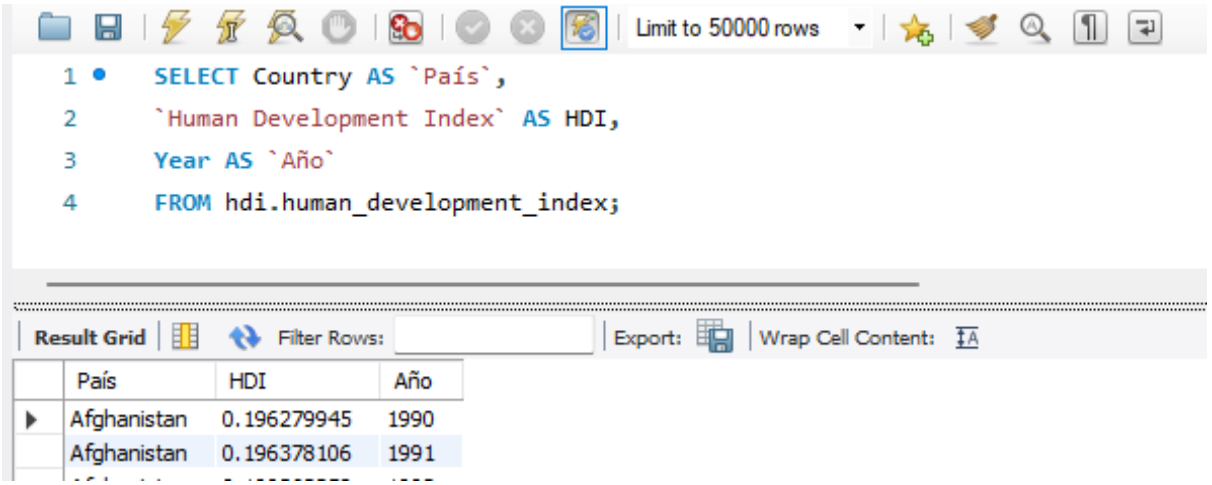


En SQL, un alias es un nombre temporal que se le da a una tabla o a una columna en el contexto de una consulta específica. Los alias son útiles por varias razones:

Legibilidad: Los alias pueden hacer que las consultas sean más fáciles de leer y entender, especialmente cuando se trabaja con tablas o columnas que tienen nombres largos o confusos.

Eficiencia: En consultas complejas que implican múltiples tablas, los alias pueden ayudar a simplificar la sintaxis y a hacer que la consulta sea más eficiente.

Renombrar: Los alias permiten renombrar columnas en el resultado de una consulta, lo cual puede ser útil para proporcionar nombres de columnas más descriptivos.



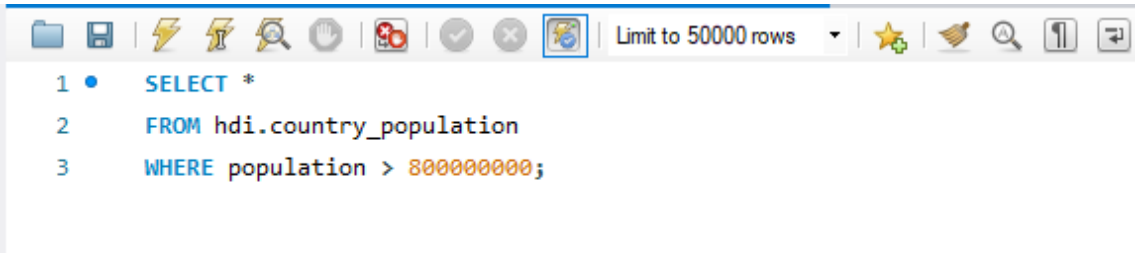
WHERE

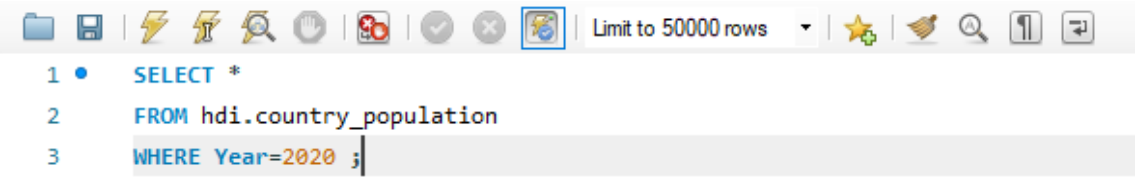
La cláusula WHERE toma el resultado devuelto por la cláusula FROM (en una tabla virtual) y aplica la condición de búsqueda que se define esta cláusula. La cláusula WHERE actúa como un filtro sobre los resultados devueltos por FROM. Cada fila se evaluará contra las condiciones especificadas en la cláusula WHERE y se devolverán las filas que se evalúan como verdaderas. Las que se evalúan como falsas no se incluirán en los resultados. En la cláusula WHERE podemos encontrar los siguientes operadores

Operador	Descripción
=	Igual a
<>	Diferente de
!=	Diferente de
<	Menor que
>	Mayor que
<=	Menor o igual que
>=	Mayor o igual que
BETWEEN	Entre un rango de valores
LIKE	Busca un patrón
IN	Compara un valor con una lista de valores
NOT	Niega una condición
AND	Evalúa dos o más condiciones y devuelve verdadero si todas son verdaderas
OR	Evalúa dos o más condiciones y devuelve verdadero si al menos una es verdadera
IS NULL	Comprueba si un valor es nulo
IS NOT NULL	Comprueba si un valor no es nulo

Operadores de Comparación*:* Los operadores de comparación son símbolos o combinaciones de símbolos que se utilizan para comparar dos valores y determinar si una determinada condición es verdadera o falsa. En SQL, los operadores de comparación se utilizan principalmente en la cláusula WHERE de una consulta SELECT para filtrar los resultados de la consulta

Los siguientes ejemplos muestran dos consultas que utiliza un operador de comparación mostrando sólo aquellos registros en que la población sea mayor de 800.000.000 o para los datos registrados en el año 2020.

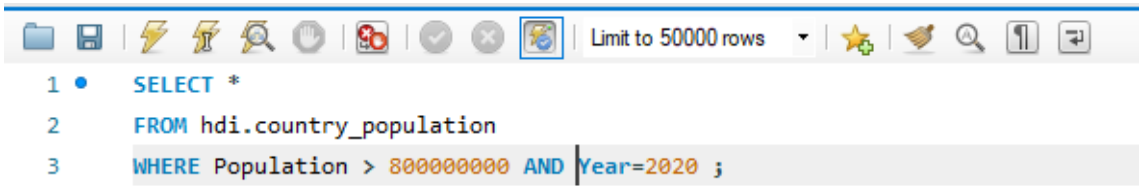




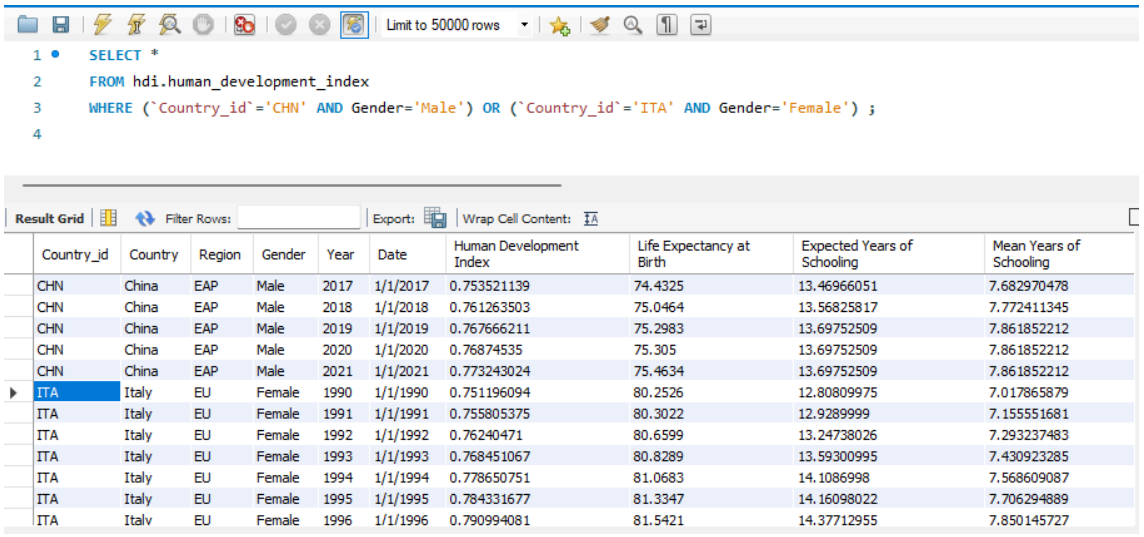
Operadores Lógicos : AND-OR: En la cláusula WHERE también podemos encontrar operadores lógicos como AND, OR y NOT. Si dos condiciones están conectadas por el operador AND, la consulta sólo devuelve resultados cuando las dos condiciones se satisfacen. Si dos condiciones están conectadas mediante el operador OR, la consulta devolverá las filas que satisfagan la primera o segunda condición (o las dos). El operador NOT cambia el valor lógico de una condición. La negación del valor VERDADERO es FALSO y viceversa. La negación de un valor NULL es también NULL.

Veamos unas consultas que utilicen estos operadores:

- En esta consulta se seleccionan todas las obras que tengan una población>8000000 y cuyo año sea 2020:

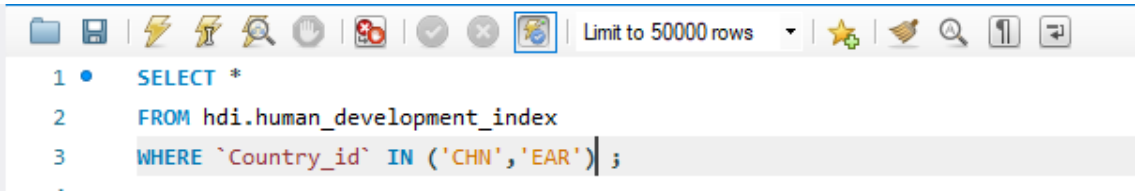


- En esta consulta se seleccionan todos los datos referentes a Hombres Chinos Y Mujeres Italianas.

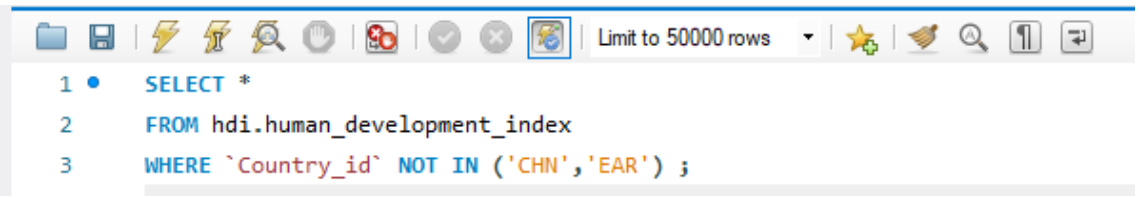


Country_id	Country	Region	Gender	Year	Date	Human Development Index	Life Expectancy at Birth	Expected Years of Schooling	Mean Years of Schooling
CHN	China	EAP	Male	2017	1/1/2017	0.753521139	74.4325	13.46966051	7.682970478
CHN	China	EAP	Male	2018	1/1/2018	0.761263503	75.0464	13.56825817	7.772411345
CHN	China	EAP	Male	2019	1/1/2019	0.767666211	75.2983	13.69752509	7.861852212
CHN	China	EAP	Male	2020	1/1/2020	0.76874535	75.305	13.69752509	7.861852212
CHN	China	EAP	Male	2021	1/1/2021	0.773243024	75.4634	13.69752509	7.861852212
ITA	Italy	EU	Female	1990	1/1/1990	0.751196094	80.2526	12.80809975	7.017865879
ITA	Italy	EU	Female	1991	1/1/1991	0.755805375	80.3022	12.9289999	7.155551681
ITA	Italy	EU	Female	1992	1/1/1992	0.76240471	80.6599	13.24738026	7.293237483
ITA	Italy	EU	Female	1993	1/1/1993	0.768451067	80.8289	13.59300995	7.430923285
ITA	Italy	EU	Female	1994	1/1/1994	0.778650751	81.0683	14.1086998	7.568609087
ITA	Italy	EU	Female	1995	1/1/1995	0.784331677	81.3347	14.16098022	7.706294889
ITA	Italy	EU	Female	1996	1/1/1996	0.790994081	81.5421	14.37712955	7.850145727

Operadores IN-OR: El operador IN permite especificar dos o más expresiones en una búsqueda. El resultado de la condición devuelve el valor VERDADERO si el valor de la columna es igual a una de las expresiones especificadas por el predicado IN. Por ejemplo, veamos la siguiente consulta:



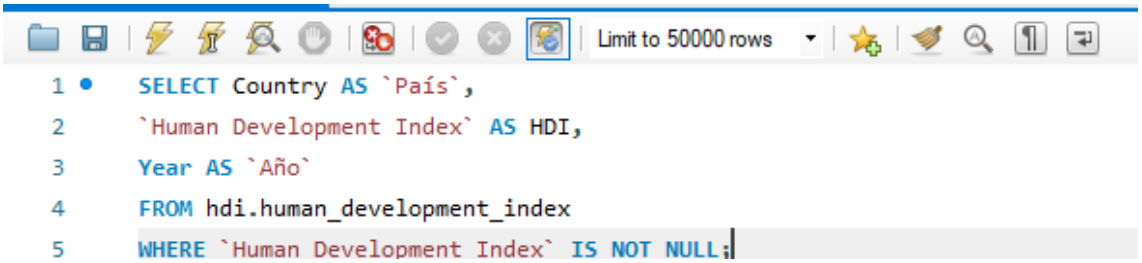
En esta consulta seleccionamos todos los registros de los con código países que son de tipo 'CHN' o 'EAR'. El operador IN equivale a una serie de condiciones conectadas por uno o más operadores OR. El operador IN se puede utilizar en combinación con el operador NOT. Veamos un ejemplo:



BETWEEN: Al contrario del operador IN que especifica cada valor individual, el operador BETWEEN especifica un rango de valores. Veamos el siguiente ejemplo:

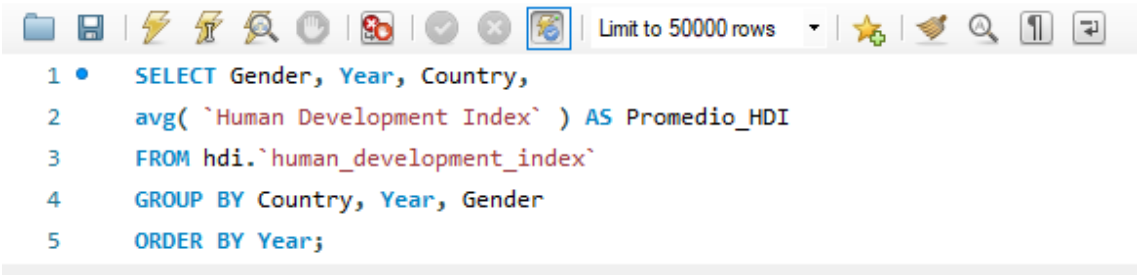


NULL La palabra clave NULL en una instrucción SELECT especifica que se permite como valor en una columna un valor especial llamado NULL. Este valor NULL se utiliza para representar que se desconoce el valor o que no es aplicable. Los valores NULL son muy distintos de los otros valores de la base de datos. La cláusula WHERE de una instrucción SELECT generalmente devuelve las filas que son VERDADERAS para la condición especificada. Nos podemos preguntar entonces cómo se evaluará una comparación cuando se encuentra con un valor NULL; todas las comparaciones con valores NULL se evaluarán como FALSAS.



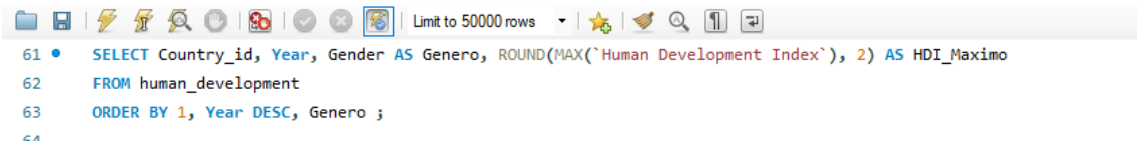
ORDER BY

La cláusula ORDER BY se utiliza para especificar el orden de las filas resultantes de una consulta. ASC indica que se quiere ordenar los datos en sentido ascendente y DESC en sentido descendente (ASC es el valor por defecto).



Para indicar las columnas por las que ordenar se puede indicar de diferentes maneras, las cuales son compatibles entre sí:

- **Nombre de columna:** especifica la columna en base a la que se ordena el resultado de la consulta.
- **Número de columna:** es una forma alternativa de especificar la columna por el número ordinal de la posición que ocupa en la lista de columnas de la cláusula SELECT.
- **ALIAS:** como alternativa se puede ordenar indicando el nombre del alias de la columna en particular:



FUNCIONES

Hasta ahora hemos trabajado con operaciones básicas para crear, insertar, consultar, modificar y eliminar datos. Sin embargo, SQL ofrece un conjunto poderoso de herramientas que nos permiten realizar cálculos, transformaciones y análisis más sofisticados: las funciones. Estas funciones incorporadas extienden significativamente las capacidades de nuestras consultas, permitiéndonos procesar y manipular datos de manera más eficiente y elegante.

Las funciones en SQL son instrucciones predefinidas que realizan operaciones específicas sobre los datos, devolviendo un resultado calculado. Pueden procesar valores individuales, realizar cálculos sobre conjuntos de registros, manipular cadenas de texto, trabajar con fechas, y mucho más. Su uso adecuado puede transformar consultas complejas en operaciones simples y legibles.

Tipos de Funciones

SQL proporciona diferentes categorías de funciones, cada una diseñada para tipos específicos de operaciones:

Funciones Escalares: Trabajan con valores individuales y devuelven un resultado por cada valor procesado. Incluyen funciones matemáticas, de cadenas y de fecha.

Funciones de Agregado: Operan sobre conjuntos de registros para producir un único valor resultado. Son fundamentales para generar reportes y análisis estadísticos.

Funciones de Cadena: Especializadas en manipular texto, permitiendo extraer, modificar, formatear y analizar datos de tipo string.

Funciones de Fecha y Hora: Diseñadas para trabajar con valores temporales, facilitando cálculos de tiempo, formateo de fechas y extracción de componentes específicos.

El uso de funciones incorporadas ofrece múltiples beneficios: mejoran la eficiencia al realizar cálculos directamente en la base de datos, reducen la cantidad de código necesario, garantizan consistencia en los resultados, y aprovechan las optimizaciones internas del motor de base de datos. Además, hacen que nuestras consultas sean más expresivas y fáciles de mantener.

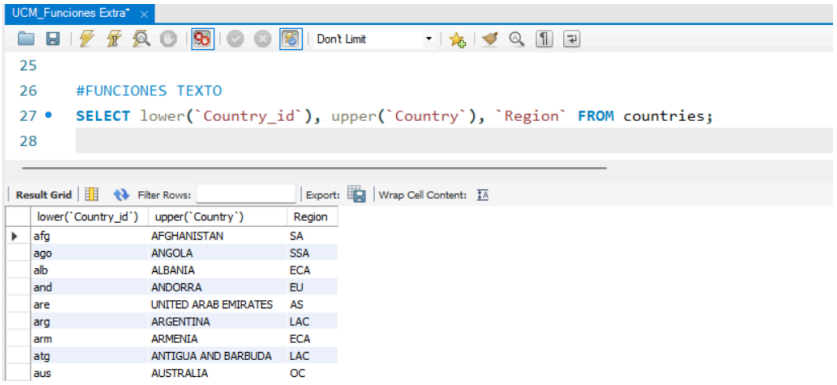
Las funciones son herramientas esenciales que todo desarrollador SQL debe dominar, ya que permiten crear consultas más potentes y obtener insights valiosos de los datos almacenados.

Funciones Escalares

Las **funciones escalares** en SQL son aquellas que operan sobre un solo valor de entrada y devuelven un solo valor como resultado. Son ampliamente utilizadas para manipular datos, realizar cálculos y transformar los valores en las consultas. En este capítulo, abordaremos las funciones escalares de **texto**, **numéricas**, y **de fecha**, así como la forma de manejar **valores nulos** en SQL.

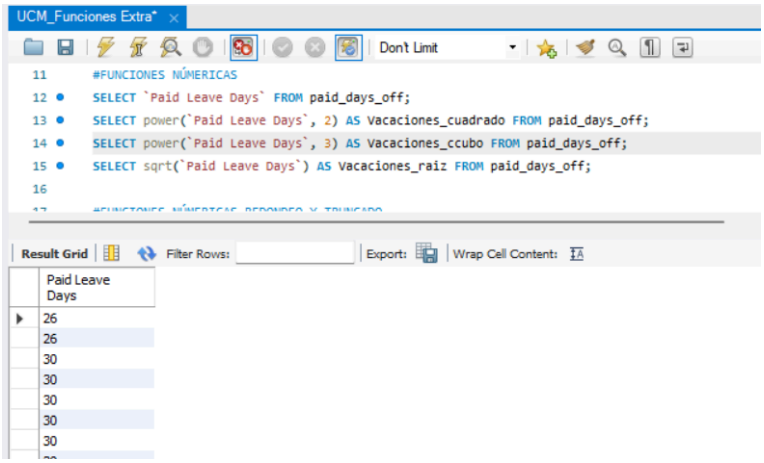
TEXTO: Las **funciones de texto** en SQL permiten manipular y formatear cadenas de texto (o **VARCHAR**, **CHAR**, entre otros tipos de datos de texto). Estas funciones son útiles para limpiar datos, realizar búsquedas, modificar cadenas o generar salidas con formato específico. A continuación, se describen algunas de las funciones más comunes:

1. **UPPER() y LOWER()**: Estas funciones convierten todas las letras de una cadena de texto a mayúsculas o minúsculas, respectivamente.
    - **UPPER(campo)**: Convierte toda la cadena de texto a mayúsculas.
    - **LOWER(campo)**: Convierte toda la cadena de texto a minúsculas.



NUMÉRICAS: Las **funciones numéricas** en SQL son aquellas que permiten realizar operaciones matemáticas sobre datos numéricos. Estas funciones son esenciales para calcular promedios, totales, o redondeos, entre otras tareas.

- **ABS()**: Devuelve el valor absoluto de un número, eliminando el signo negativo si lo tiene.
- **ROUND()**: Redondea un número a un número específico de decimales.
- **CEIL()**: Redondea un número hacia arriba, al siguiente número entero.
- **FLOOR()**: Redondea un número hacia abajo, al siguiente número entero inferior.
- **MOD()**: Devuelve el resto de la división entre dos números.
- **POWER()**: Eleva un número a la potencia de otro número.
- **SQRT()**: Devuelve la raíz cuadrada de un número.
- **EXP()**: Calcula el valor de "e" elevado a la potencia de un número.
- **LOG()**: Devuelve el logaritmo natural de un número.



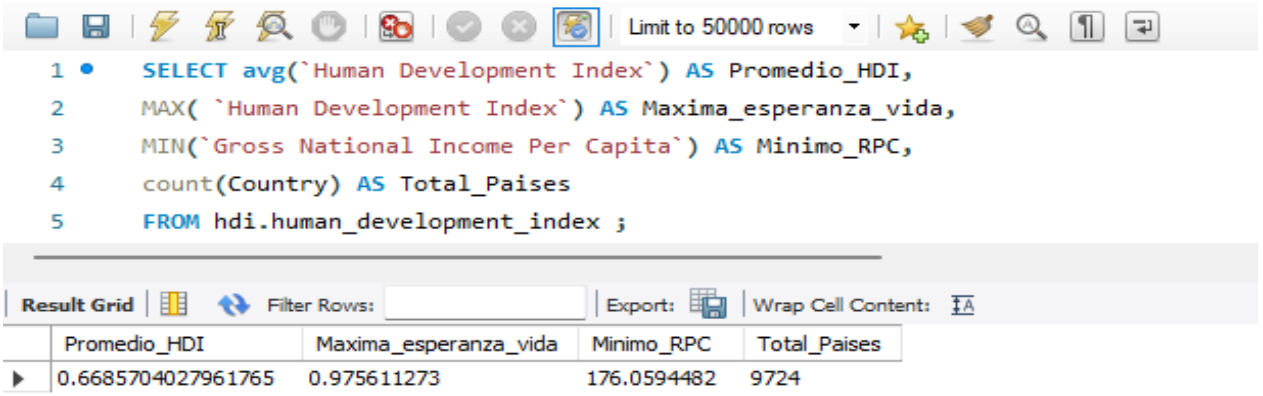
NULOS: En SQL, los **valores nulos (NULL)** representan la ausencia de un valor. Las funciones y operadores específicos permiten manejar estos casos sin causar errores en las consultas.

- **IS NULL**: Verifica si un valor es NULL.
- **IS NOT NULL**: Verifica si un valor no es NULL.
- **COALESCE()**: Devuelve el primer valor no nulo de una lista de expresiones.

Funciones Agregación

Las funciones de agregación en SQL son funciones que toman múltiples filas como entrada y devuelven un solo valor. Son extremadamente útiles cuando necesitas realizar cálculos en tus datos. Aquí tienes un resumen de las más comunes:

- **COUNT():** Esta función devuelve el número de filas que coinciden con un criterio especificado.
- **SUM():** Esta función devuelve la suma total de una columna numérica.
- **AVG():** Esta función devuelve el valor promedio de una columna numérica.
- **MIN() y MAX():** Estas funciones devuelven el valor mínimo y máximo de una columna, respectivamente.



```
1  SELECT avg(`Human Development Index`) AS Promedio_HDI,
2  MAX( `Human Development Index`) AS Maxima_esperanza_vida,
3  MIN(`Gross National Income Per Capita`) AS Minimo_RPC,
4  count(Country) AS Total_Paises
5  FROM hdi.human_development_index ;
```

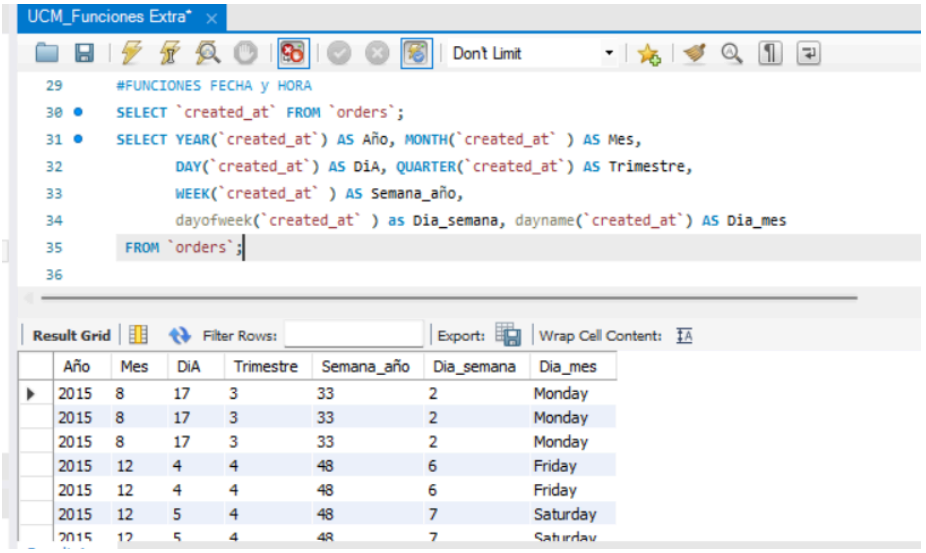
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Promedio_HDI	Maxima_esperanza_vida	Minimo_RPC	Total_Paises
0.6685704027961765	0.975611273	176.0594482	9724

Funciones de Fecha

Las funciones de fecha en SQL permiten trabajar con datos de fecha y hora, facilitando la extracción, manipulación y cálculo de fechas y tiempos.

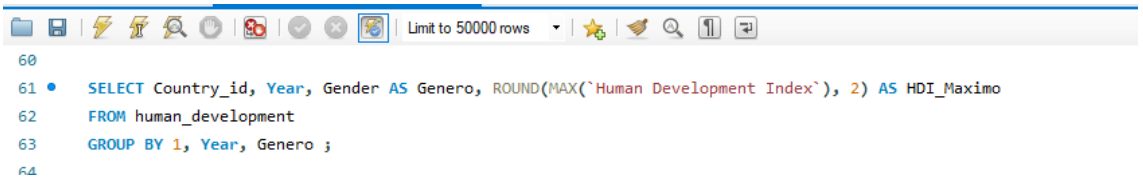
- NOW(): Devuelve la fecha y hora actuales del sistema.
- CURDATE(): Devuelve la fecha actual del sistema.
- CURTIME(): Devuelve la hora actual del sistema.
- DATEDIFF(): Devuelve la diferencia entre dos fechas en días.
- DATE_ADD(): Añade un intervalo de tiempo específico (días, meses, años) a una fecha.
- DATE_SUB(): Resta un intervalo de tiempo específico a una fecha.
- YEAR(): Extrae el año de una fecha.
- MONTH(): Extrae el mes de una fecha.
- DAY(): Extrae el día de una fecha.



GROUP BY

La cláusula GROUP BY define una o más columnas como un grupo, de forma que todas las filas de cualesquiera de los grupos tienen los mismos valores para todas las columnas. Para indicar las columnas a agrupar se puede indicar de diferentes maneras, las cuales son compatibles entre sí:

- **Nombre de columna:** especifica la columna en base a la que se agrupa la función de agregación de la consulta.
- **Número de columna:** es una forma alternativa de especificar la columna por la que agrupar por el número ordinal de la posición que ocupa en la lista de columnas de la cláusula SELECT.
- **ALIAS:** como alternativa se puede agrupar indicando el nombre del alias de la columna en particular:



En esta consulta se quiere agrupar la consulta en base a la región y el país. La consulta devuelve todos los posibles grupos basados en los valores de la columna type.

Una tabla se puede agrupar por cualquier combinación de columnas:

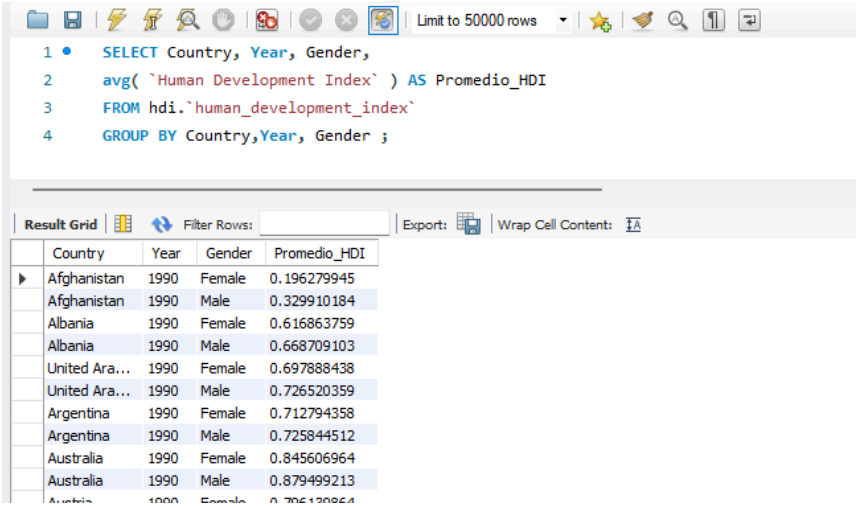
```
1 •  SELECT Country, Region, ROUND(avg(`Human Development Index`), 2) AS Promedio_HDI
2    FROM hdi.human_development_index
3    GROUP BY Country, Region ;
4
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Country	Region	Promedio_HDI
Afghanistan	SA	0.37
Angola	SSA	0.5
Albania	ECA	0.71
United Arab Emirates	AS	0.81
Argentina	LAC	0.8

La granularidad en las bases de datos se refiere al nivel de detalle de los datos almacenados. Un nivel de granularidad más alto significa que los datos se desglosan en detalles más pequeños. Por ejemplo, si tienes una base de datos de ventas, un alto nivel de granularidad podría implicar tener datos detallados para cada transacción individual. Por otro lado, un nivel de granularidad más bajo significa que los datos se agrupan en unidades más grandes. En el mismo ejemplo de la base de datos de ventas, un bajo nivel de granularidad podría ser tener sólo el total de ventas por día o por mes.

Por lo tanto, puedes considerar la granularidad como una medida del nivel de detalle en una base de datos. Es un equilibrio entre tener suficiente detalle para realizar análisis significativos y no tener tanto detalle que la base de datos se vuelva demasiado grande y difícil de manejar. La elección del nivel de granularidad adecuado dependerá del caso específico y del tipo de análisis que se quiera realizar.



```
1  SELECT Country, Year, Gender,
2  avg( `Human Development Index` ) AS Promedio_HDI
3  FROM hdi.`human_development_index`
4  GROUP BY Country,Year, Gender ;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Country	Year	Gender	Promedio_HDI
Afghanistan	1990	Female	0.196279945
Afghanistan	1990	Male	0.329910184
Albania	1990	Female	0.616863759
Albania	1990	Male	0.668709103
United Ara...	1990	Female	0.697888438
United Ara...	1990	Male	0.726520359
Argentina	1990	Female	0.712794358
Argentina	1990	Male	0.725844512
Australia	1990	Female	0.845606964
Australia	1990	Male	0.879499213

b. Combinación de Tablas

La operación JOIN nos permite el acceso a varias tablas a la vez. Esta operación es muy útil cuando se desea consultar datos relacionados de más de una tabla y recuperarlos de una forma en que las relaciones entre las tablas sean invisibles en la práctica. La operación JOIN hace coincidir las filas de una tabla con las filas de otra tabla de forma que las columnas de ambas tablas se puedan colocar unas al lado de otras en los resultados de la consulta como si vinieran de una sola tabla.

Existen dos sintaxis distintas para realizar una operación JOIN:

- **Sintaxis explícita:** de acuerdo al estándar ANSI SQL:1992
- **Sintaxis implícita:** esta sintaxis se considera anticuada

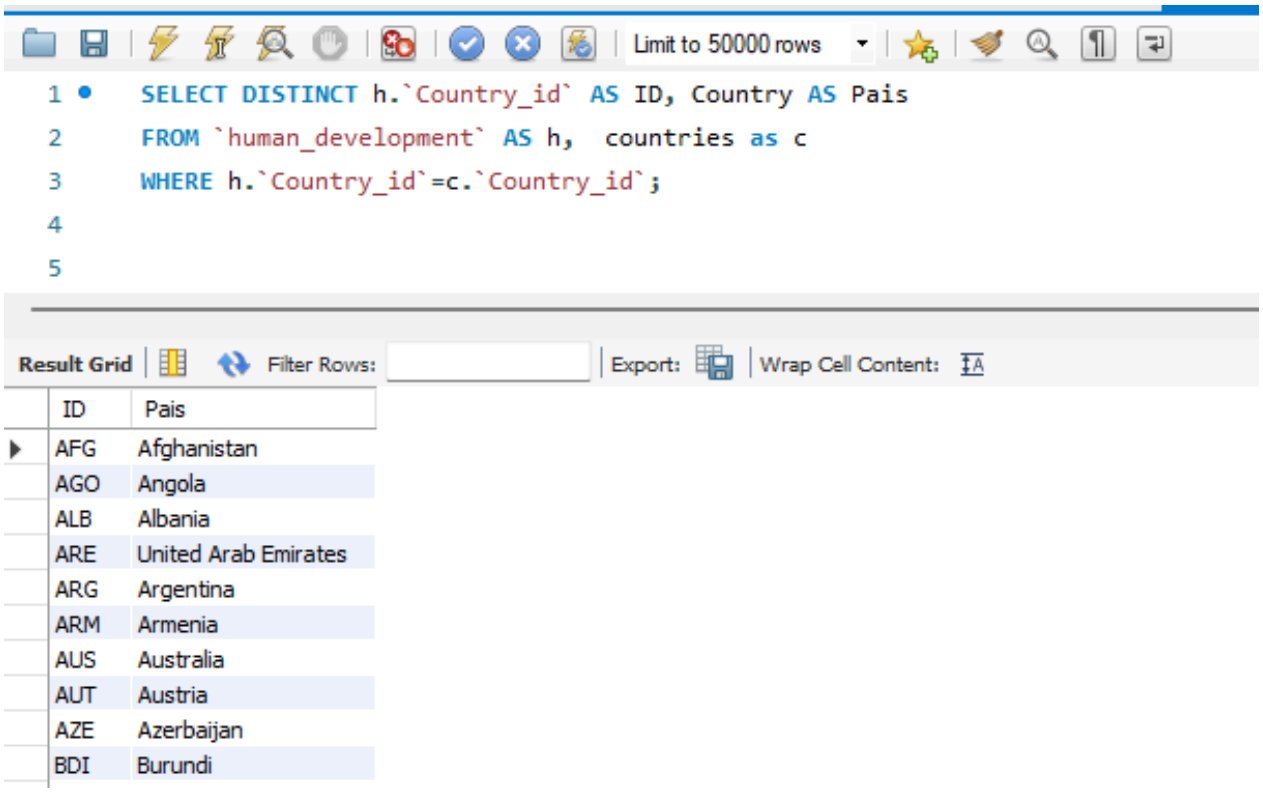
Se prefiere la utilización de la **sintaxis explícita** que se introdujo en el estándar SQL92 donde se definen las operaciones JOIN de forma explícita, es decir, se asigna un nombre a cada tipo de operación JOIN. Encontramos las siguientes palabras clave en SQL Server que representan los distintos tipos de operaciones JOIN:

- [INNER] JOIN
- LEFT [OUTER] JOIN
- RIGHT [OUTER] JOIN
- FULL [OUTER] JOIN
- CROSS JOIN

Sintaxis implícita: JOIN NATURAL

En las operaciones JOIN naturales se equiparan los valores de una o más columnas en la primera tabla con los valores correspondientes de la segunda tabla. Para realizar una operación JOIN es necesario especificar las columnas donde se realiza la comparación de ambas tablas.

Por ejemplo, veamos una consulta donde se realiza un join natural entre dos tablas:

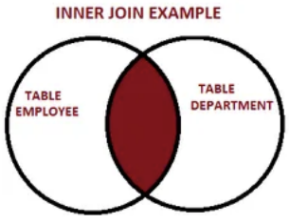


ID	Pais
AFG	Afghanistan
AGO	Angola
ALB	Albania
ARE	United Arab Emirates
ARG	Argentina
ARM	Armenia
AUS	Australia
AUT	Austria
AZE	Azerbaijan
BDI	Burundi

En esta consulta se utiliza la sintaxis implícita de join. La consulta recupera los datos de los índices de todos los países y la población correspondiente. Nótese que se ha definido un alias para la tabla *human_development_index* (h) y para la tabla *country_population* (p). El alias se puede utilizar en la consulta para referirse a esa tabla en concreto en vez de escribir. el nombre completo de la tabla. En esta consulta se especifican en la cláusula FROM las tablas que se van a unir y el tipo de join que se va a utilizar (INNER JOIN).

En la cláusula WHERE se especifica las columnas que se van a utilizar para realizar la operación join. En esta consulta se devuelven las filas de la tabla *human_developmet_index* donde la columna country_id coincida con la columna id de la tabla *country_population.* Esta consulta se puede reescribir usando la sintaxis explícita de la siguiente forma:

Sintaxis explícita



INNER JOIN

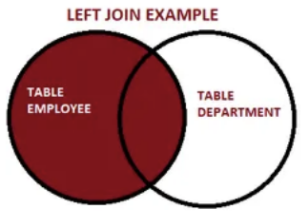
Esta consulta se puede reescribir usando la sintaxis explícita de la siguiente forma:

```
7  SELECT DISTINCT h.`Country_id` AS ID, Country AS Pais
8  FROM `human_development` AS h INNER JOIN countries as c
9  ON h.`Country_id`=c.`Country_id`;
10
11
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID	Pais
AFG	Afghanistan
AGO	Angola
ALB	Albania
ARE	United Arab Emirates
ARG	Argentina
ARM	Armenia
AUS	Australia
AUT	Austria
AZE	Azerbaijan
BDI	Burundi
BEL	Belgium
BEN	Benin
BFA	Burkina Faso
BGD	Bangladesh

En esta consulta se utiliza la cláusula ON para especificar la condición en la que se basa la combinación. La condición puede especificar cualquier predicado, aunque se suelen utilizar columnas y operadores de comparación. Se devuelven las filas de la tabla *human_developmet_index* donde la columna country_id coincida con la columna id de la tabla *country_population*.



LEFT JOIN

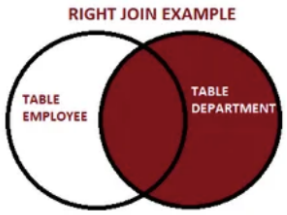
Cuando hacemos un LEFT JOIN, estamos trayendonos todos los datos de la tabla situada en la izquierda (FROM) y aquellos registros de la tabla de la derecha (JOIN) que tienen valores coincidentes en los campos indicados en el ON:

```
21 •  SELECT Country_id AS ID, h.Year, `Human Development Index` AS HDI, Population AS Poblacion
22    FROM `human_development` AS h LEFT JOIN `country_population` AS c
23    ON h.`Country_id`=c.`country ID` AND h.year=c.year;
24
25
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID	Year	HDI	Poblacion
WSM	2021	0.716499047	NULL
YEM	2021	0.262569632	NULL
YEM	2021	0.529016004	NULL
ZAF	2021	0.685864456	NULL
ZAF	2021	0.726844169	NULL
ZMB	2021	0.554427499	NULL
ZMB	2021	0.574296636	NULL
ZWE	2021	0.580303186	NULL
ZWE	2021	0.604054039	NULL
AFG	2020	0.3740333	38928341
AFG	2020	0.542963575	38928341
AGO	2020	0.560780317	32866268
AGO	2020	0.620493506	32866268
ALB	2020	0.79657131	2837743
ALB	2020	0.791941236	2837743
ARE	2020	0.877874432	9890400
ARE	2020	0.921901043	9890400
ARG	2020	0.831992596	45376763
ARG	2020	0.832783696	45376763

En esta consulta se devuelven las filas de la tabla *human_developmet_index* donde la columna country_id coincida con la columna id de la tabla population y además todas aquellas filas de la tabla *human_developmet_index* que no coincidan (donde no haya referencia con *country_population*) devolviendo nulos para la tabla population como en el caso del año 2021, donde no se tiene datos de población



RIGHT JOIN

Cuando hacemos un RIGHT JOIN, estamos trayéndonos las filas que tienen valores coincidentes en los campos indicados en el ON de la tabla situada en la izquierda (FROM) y todos los registros de la tabla de la derecha (JOIN):

```
25 •  SELECT c.`country ID`AS ID, c.Year, `Human Development Index` AS HDI, Population AS Poblacion
26    FROM `human_development` AS h RIGHT JOIN `country_population` AS c
27    ON h.`Country_id`=c.`country ID` AND h.year=c.year;
28
29    #MULTIPLES JOIN
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID	Year	HDI	Poblacion
ABW	1960	NULL	54208
AFE	1960	NULL	130836765
AFG	1960	NULL	8996967
AFW	1960	NULL	96396419
AGO	1960	NULL	5454938
ALB	1960	NULL	1608800
AND	1960	NULL	13410
ARB	1960	NULL	92197715
ARE	1960	NULL	92417
ARG	1960	NULL	20481781
ARM	1960	NULL	1874119
ASM	1960	NULL	20127
ATG	1960	NULL	54132
AUS	1960	NULL	10276477
AUT	1960	NULL	7047539
AZE	1960	NULL	3895398
BDI	1960	NULL	2797925
BEL	1960	NULL	9153489

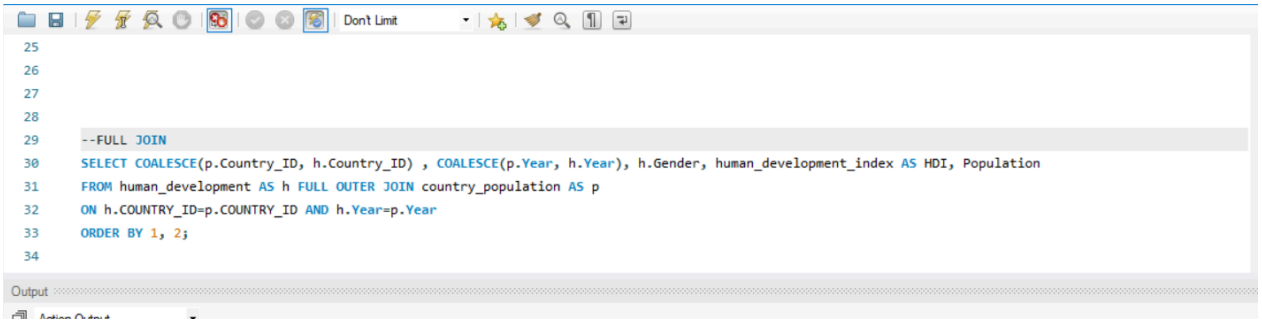
En esta consulta se devuelven las filas de la tabla *human_developmet_index* donde la columna country_id coincida con la columna id de la tabla population y además todas aquellas filas de la tabla *country_population* que no coincidan (donde no haya referencia con *human_developmet_index*) devolviendo nulos para la tabla *human_developmet_index* como en el caso de los años 1960-1989, donde no se tiene datos de HDI

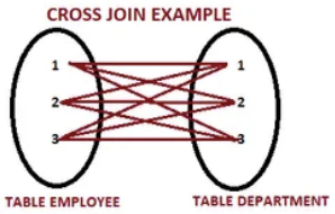


FULL JOIN

Cuando hacemos un FULL OUTER JOIN, estamos trayendonos todos los datos de la tabla situada en la izquierda (FROM) y todos los de la tabla de la derecha (JOIN), tengan o no valores coincidentes en los campos indicados en el ON:

En esta consulta se devuelven las filas de la tabla *human_developmet_index* donde la columna country_id coincida con la columna id de la tabla *country_population* y además todas aquellas filas de las dos tablas que no coincidan.





CROSS JOIN

El CROSS JOIN en SQL devuelve el producto cartesiano de dos tablas, es decir, combina cada fila de la primera tabla con cada fila de la segunda tabla. Esto significa que si la primera tabla tiene *m* filas y la segunda tabla tiene *n* filas, el resultado del CROSS JOIN tendrá *m* × *n* filas. El CROSS JOIN es útil en situaciones donde se requiere emparejar todas las filas de una tabla con todas las filas de otra tabla, generando combinaciones posibles sin aplicar ninguna condición.

c. Consultas avanzadas

SUBCONSULTAS

Hasta ahora hemos visto consultas donde se compara valores de columna con expresiones o constantes. El lenguaje de consulta SQL también ofrece la habilidad de comparar valores de columna con el resultado de una consulta SELECT. Esta construcción, donde una o más consultas SELECT están anidadas en la cláusula WHERE de otra consulta SELECT, se llama subconsulta. La primera consulta SELECT de una subconsulta se llama consulta externa y la consulta anidada recibe el nombre de consulta interna. La consulta interna se evalúa primero y la consulta externa recibe los valores de la consulta interna.

Existen dos tipos de subconsultas:

- Subconsultas independientes
- Subconsultas correlacionadas

Subconsulta independiente

En una subconsulta independiente la consulta interna se evalúa sólo una vez. En una subconsulta correlacionada su valor depende de una variable de la consulta externa y debido a esto la consulta interna se evalúa cada vez que el sistema recupera una fila nueva de la consulta externa.

Una subconsulta independiente se puede usar con los siguientes operadores:

- Operadores de comparación
- Operador IN

El operador IN se utiliza en una subconsulta cuando se quiere hacer una comparación con un conjunto de valores. Veamos el siguiente ejemplo donde se recupera todas los países con una población superior a 500.0000.000

Primero se evalúa la consulta interna que devuelve un conjunto de valores, en este caso, todas los países con una población superior al valor indicado. Esta consulta es equivalente a escribir la consulta de esta manera:

```
SELECT Country, Gender, `Human Development Index`
FROM hdi.`human_development_index`
WHERE
YEAR=2020 AND `Country_id` IN
(SELECT `country ID`
FROM hdi.`country_population`
WHERE year = 2020 AND Population > 500000000
 );
```

Veamos otro ejemplo de una subconsulta con el operador IN para devolver sólo las columnas de aquellos países con más de 43 días de vacaciones totales por ciudadano:

```
SELECT Country, Gender, `Human Development Index`
FROM hdi.`human_development_index`
WHERE
YEAR=2020 AND `Country_id` IN
(SELECT `country ID`
FROM hdi.`paid_days_off`
WHERE `Paid Leave Days` + `Paid Public Holidays` >43
ORDER BY `Paid Leave Days` + `Paid Public Holidays` );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Country	Gender	Human Development Index
Bahrain	Female	0.830807522
Bahrain	Male	0.895332486
Bhutan	Female	0.642684491
Bhutan	Male	0.685985086
Iran (Islamic Republic of)	Female	0.704607265
Iran (Islamic Republic of)	Male	0.802689023
Yemen	Female	0.272683861
Yemen	Male	0.535355202

Subconsulta relacionadas

La sub-queries relacionadas se refieren a que cuando seguimos la estructura básica de las queries SELECT * FROM TABLE

La TABLE puede ser a su vez una Query y que la consulta SELECT * FROM se haga sobre el resultado de una query previa. Podemos realizar tantas queries anidadas como sea necesario. De esta manera la sintaxis sería:

SELECT * FROM (SELECT * FROM TABLE)

```
Limit to 50000 rows
1  SELECT Country, Promedio_HDI_2020
2  FROM
3  (SELECT Country, ROUND(avg(`Human Development Index`), 2)AS Promedio_HDI_2020
4  FROM hdi.`human_development_index`
5  WHERE Year = 2020
6  GROUP BY Country
7   ) AS sq1;
```

Siguiendo el ejemplo de las subqueries relacionadas podemos usarlo para también para la agregación de tablas usando JOINS, combinando únicamente los sets de datos que queramos visualizar en el insight e independientemente de que tengan diferentes niveles de agregación:

```
Limit to 50000 rows
1  SELECT `country ID`,
2  Promedio_HDI_Genero,
3  Promedio_Poblacion
4  FROM
5  (SELECT `Country_id`, ROUND(avg(`Human Development Index`), 2) AS Promedio_HDI_Genero
6  FROM hdi.`human_development_index`
7  GROUP BY `Country_id`, Gender
8   ) AS S1
9   JOIN
10 (SELECT `country ID`,  ROUND(AVG(Population), 2) AS Promedio_Poblacion
11 FROM hdi.`country_population`
12 GROUP BY `country ID`) as S2
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

country ID	Promedio_HDI_Genero	Promedio_Poblacion
AFG	0.28	18699394.44
AFG	0.46	18699394.44
AGO	0.47	14135453.72
AGO	0.54	14135453.72
ALB	0.7	2710419.23
ALB	0.73	2710419.23
ARE	0.78	3200561.75
ARE	0.83	3200561.75
ARG	0.79	32515550.69

Result 49

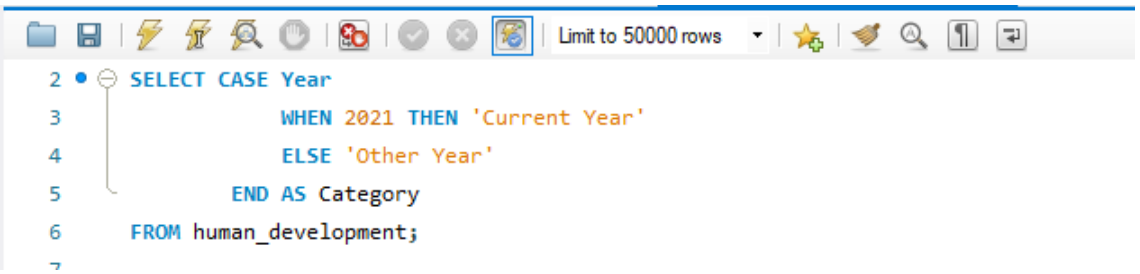
CASE WHEN

La expresión CASE WHEN en SQL es una herramienta que permite realizar evaluaciones condicionales en una consulta SELECT. La sintaxis básica de la expresión CASE WHEN incluye la palabra clave CASE, la palabra clave WHEN, las condiciones y los resultados correspondientes, y la palabra clave ELSE para establecer un resultado por defecto. La expresión CASE WHEN se puede utilizar en combinación con otras cláusulas de SQL para realizar evaluaciones condicionales más complejas.

Hay dos tipos de expresiones CASE: simple y de búsqueda

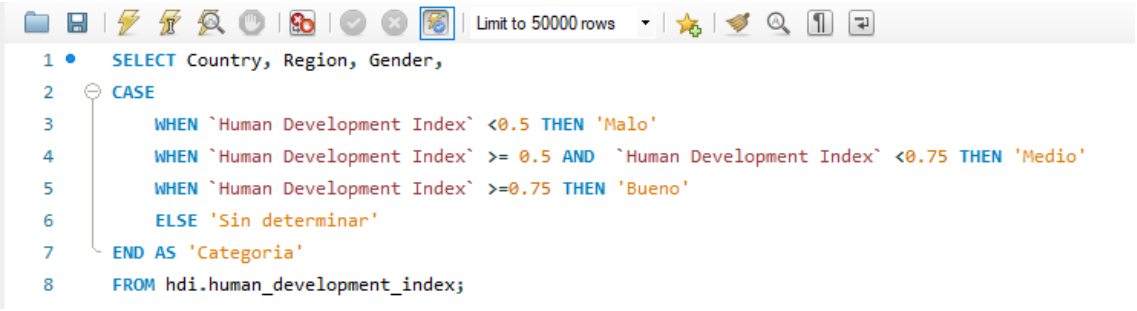
Expresión CASE simple

Una consulta SQL con una expresión CASE simple busca la primera expresión de la lista de cláusulas WHEN que coincide con expresión_1. Se devolverá la expresión que se encuentra a la derecha de la cláusula THEN correspondiente. Si no hay ninguna coincidencia se evalúa la parte ELSE. Ejecute el siguiente ejemplo donde identificamos el HDI para el último año como “Last Index” y el resto de años como “Previous Index”.



Expresión CASE de búsqueda

En este tipo de instrucción CASE busca la primera expresión que se evalúe a VERDADERA. Si ninguna de las condiciones WHEN se evalúan como VERDADERA, se devuelve el valor de la parte ELSE. Veamos un ejemplo donde se clasifican los HDI en tres categorías dependiendo de su valor. En caso de que no se cumpla ninguna de las condiciones se asigna el valor 'Sin determinar'.



VISTAS

Una vista es una tabla virtual cuya definición existe como un objeto en la base de datos. Las vistas se crean para ver datos específicos de una o varias tablas y a diferencia de las tablas persistentes, en las vistas no se almacenan los datos. Las vistas se crean a partir de una consulta SQL que devuelve datos. Una vez que se ha creado la vista se puede seleccionar datos de ella llamándola por su nombre como si fuera una tabla normal.

Una de las ventajas de las vistas es que se pueden utilizar para definir y almacenar consultas complejas. En vez de crear las consultas cada vez que se necesiten, se puede invocar la vista. Las vistas también se utilizan para presentar a los usuarios la información que necesitan, y así mantener oculta la información que no necesitan o no deben ver. Esto puede ser especialmente relevante para ocultar información confidencial como sueldos de empleados o números de cuenta. Con una vista podríamos ocultar este tipo de columnas.

La sintaxis básica de la instrucción CREATE VIEW es la siguiente:

- nombre_vista es el nombre que se le asigna a la vista. Este nombre se usará como el nombre de una tabla en una consulta.
- lista_columnas es la lista de columnas de la vista. Este parámetro es opcional, si no se especifica se infiere de la consulta SQL.
- consulta_sql es la instrucción SELECT que devuelve las filas y columnas de una o más tablas (o vistas).

Vemos un ejemplo con esta consulta que utiliza la expresión CASE WHEN para crear una nueva columna llamada "Category" que clasifica los países según su índice de desarrollo humano promedio y el número de días de vacaciones pagadas. La consulta utiliza una subconsulta para calcular el índice de desarrollo humano promedio y el número de días de vacaciones pagadas para cada país entre 1990 y 1999. Luego, la expresión CASE WHEN se utiliza para clasificar los países en tres categorías: "Top Country", "Medium Country" y "Bottom Country", según su índice de desarrollo humano promedio y el número de días de vacaciones pagadas. Si un país no cumple ninguna de las condiciones anteriores, se clasifica como "Other". La consulta SELECT devuelve las columnas "Country_id", "Promedio_HDI", "Days_off" y "Category". Esta consulta la almacenamos como vista que llamamos *count_category* y que podremos en el futuro.

```sql
CREATE VIEW Count_Category AS
SELECT `Country_id`, Promedio_HDI, Days_off,
CASE
    WHEN Promedio_HDI < 0.8 AND Days_off >20  THEN "Top Country"
    WHEN Promedio_HDI > 0.7 AND Promedio_HDI <= 0.8 AND Days_off >15  THEN "Medium Country"
    WHEN Promedio_HDI >= 0.7 THEN "Bottom Country"
    ELSE "Other"
END AS Category
FROM (SELECT `Country_id`, ROUND(AVG(`Human Development Index`), 2) AS Promedio_HDI, AVG(`Paid Leave Days`) AS Days_off
      FROM human_development AS i INNER JOIN paid_days_off AS p
      ON i.`Country_id`=p.`country ID`
      WHERE i.Year Between 1990 AND 1999 AND `Human Development Index` IS NOT NULL
      GROUP BY 1) S1;

SELECT Category, COUNT(DISTINCT `Country_id`) Conteo_paises FROM human_study.count_category
GROUP BY 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Category	Conteo_paises
Bottom Country	24
Medium Country	15
Other	56
Top Country	35

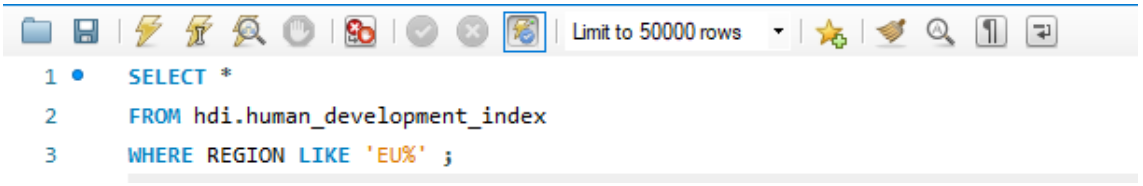
LIKE

El operador LIKE se usa para implementar una búsqueda por patrones, es decir, compara el valor de una columna con un patrón. El tipo de datos de la columna puede ser alfanumérico o una fecha. La sintaxis general es la siguiente:

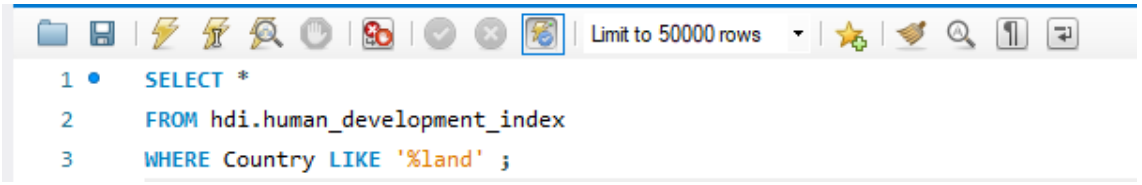
El patrón tiene que ser una constante o expresión alfanumérica o de fecha y tiene que ser compatible con el tipo de datos de la columna correspondiente. La comparación entre el valor de una columna y el patrón se evalúa como VERDADERA si el valor coincide con la expresión del patrón.

Algunos caracteres tienen un significado específico en un patrón:

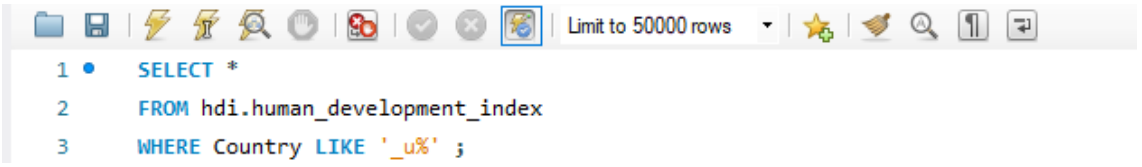
- **% (porcentaje),** especifica cualquier secuencia de cero o más caracteres
    - Ejemplo 1: Mostrar toda la información de las región que comienzan por EU



- Ejemplo 2: Mostrar toda la información de las región que terminan en -land



- **_ (guión bajo)**, especifica un solo carácter.
  - Ejemplo 3: Mostrar toda la información de las región que terminan en de los países cuya segunda letra es una “u”

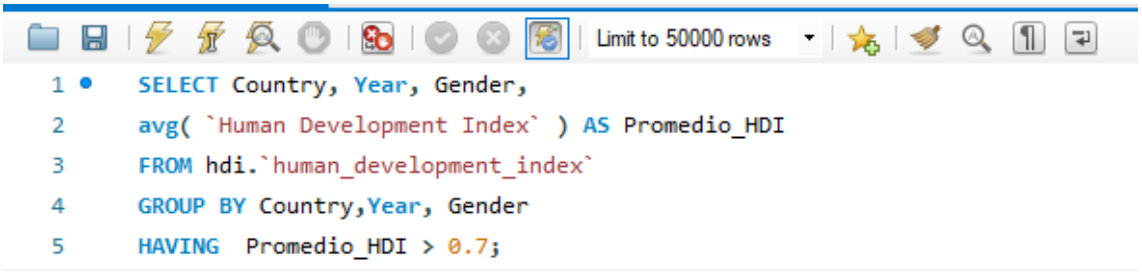


TOP N

La función TOP n en SQL se utiliza para limitar el número de filas devueltas por una consulta. Es comúnmente utilizada cuando se desea obtener sólo una cantidad específica de resultados en lugar de todos los registros de una tabla. Esta función es útil en situaciones donde se requieren sólo las primeras filas de un conjunto de resultados ordenado, como los primeros 10 resultados en una lista de productos o clientes.

HAVING

La cláusula HAVING es similar a la cláusula WHERE en cuanto a que define una condición de búsqueda. Sin embargo, la cláusula HAVING se utiliza para referirse a grupos y no a filas individuales. La cláusula HAVING se suele usar en combinación con la cláusula GROUP BY, pero también se puede usar sin ella. En ese caso todas las filas de la tabla pertenecen al mismo grupo. Veamos un ejemplo, en esta consulta seleccionamos los países que tengan un HDI mayor de 0.7.



SET OPERATORS:

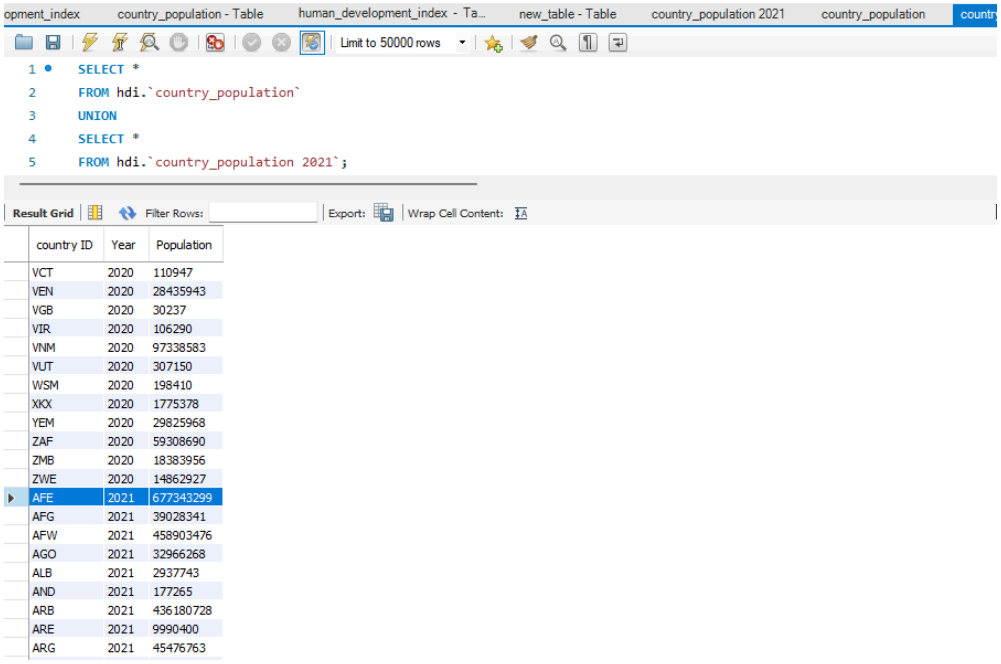
UNION

El operador UNION implementa la unión de dos conjuntos en uno solo. La unión de dos tablas genera una nueva tabla que contiene todas las filas que aparecen en una o ambas tablas. La sintaxis es la siguiente:

Si se usa la opción ALL se mostrarán todas las filas resultantes, incluyendo duplicados. La palabra clave ALL tiene el mismo significado con la cláusula UNION que con la cláusula SELECT. La única diferencia es que no es la opción por defecto para UNION pero sí lo es para SELECT.

Dos tablas se pueden unir mediante el operador UNION si son compatibles. Esto quiere decir que las dos listas de columnas tienen que tener el mismo número de elementos y los tipos de datos tienen que ser compatibles (por ejemplo, INT y SMALLINT son tipos de datos compatibles). Sólo se puede ordenar el resultado de una UNION si la cláusula ORDER BY se usa en el último SELECT.

Para este ejemplo usaremos la data referente a la población en el año 2021 la cual se recoge en la tabla *country_ population _2021*. Usamos el set operator UNION para obtener los datos de población de todos los años en nuestra BBDD, incluído 2021:



Funciones de Ventana

Una ventana es un grupo de filas relacionadas. Por ejemplo, una ventana podría definirse en función de las marcas de tiempo, con todas las filas del mismo mes agrupadas en la misma ventana. O se puede definir una ventana según la ubicación, con todas las filas de una ciudad en particular agrupadas en la misma ventana.

Una ventana puede constar de cero, una o varias filas. Para simplificar, la documentación de Snowflake suele decir que una ventana contiene varias filas.

A una función de ventana generalmente se le pasan dos parámetros:

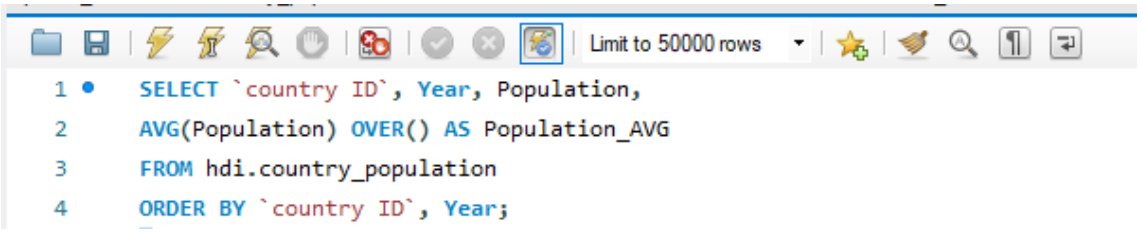
- **Una fila.** Más precisamente, a una función de ventana se le pasan 0 o más expresiones. En casi todos los casos, al menos una de esas expresiones hace referencia a una columna en esa fila. (La mayoría de las funciones de ventana requieren al menos una columna o expresión, pero algunas funciones de ventana, como algunas funciones relacionadas con el rango, no requieren una columna o expresión explícita).
- **Una ventana de filas relacionadas que incluye esa fila.** La ventana puede ser la tabla completa o un subconjunto de las filas de la tabla.

Para las funciones que no son de ventana, todos los argumentos generalmente se pasan explícitamente a la función, por ejemplo:

MI_FUNCIÓN(argumento1, argumento2, …)

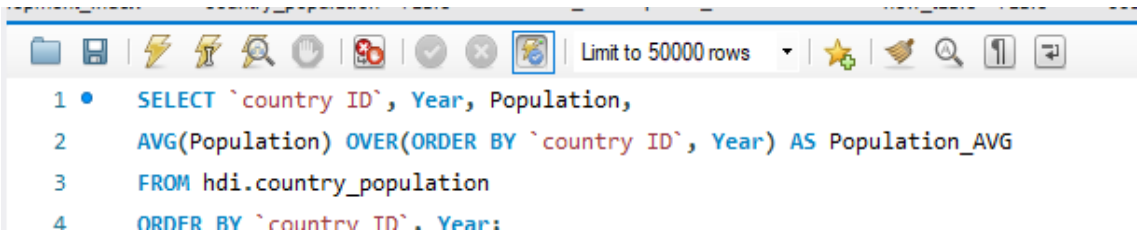
Las funciones de ventana se comportan de manera diferente; aunque la fila actual se pasa como argumento de la manera normal, la ventana se pasa a través de una cláusula separada, llamada cláusula OVER. La sintaxis de la cláusula OVER se documenta más adelante.

Una función de ventana agregada es similar a una función agregada ordinaria, excepto que agregarla a una consulta no cambia la cantidad de filas devueltas. En cambio, para cada fila, el resultado de la función de ventana agregada es como si el agregado correspondiente se ejecutará en todas las filas en el "marco de ventana" especificado por la cláusula OVER.



En este caso, como no se ha especificado nada en la función OVER(), se realiza la misma agregación para toda la base de datos.

Vamos a ver un ejemplo en que indicamos una ordenación dentro de la función OVER():



Con el fin de calcular las funciones de la ventana, el conjunto de resultados de una consulta se divide en una o más "particiones". Una partición consta de todas las filas que tienen el mismo valor para todos los términos de la cláusula PARTITION BY en la definición de ventana. Si no hay una cláusula PARTITION BY, todo el conjunto de resultados de la consulta es una sola partición. El procesamiento de la función de ventana se realiza por separado para cada partición.

```
-- Función de ventana sin partición ni ordenación: Suma total del salario para toda la tabla
SELECT nombre,
       SUM(salario) OVER() AS total_salarial
FROM empleados;

-- Función de ventana con ordenación: Salario acumulado para cada empleado
SELECT nombre,
       salario,
       SUM(salario) OVER(ORDER BY salario DESC) AS acumulado_salarial
FROM empleados;

-- Función de ventana con PARTITION BY: Salario total por departamento
SELECT id_departamento,
       nombre,
       SUM(salario) OVER(PARTITION BY id_departamento ORDER BY salario DESC) AS total_salarial_departamento
FROM empleados;
```

COMMON TABLE EXPRESSIONS- CTE

A medida que nuestras consultas se vuelven más complejas, a menudo necesitamos trabajar con resultados intermedios o crear consultas anidadas que pueden volverse difíciles de leer y mantener. Las Common Table Expressions (CTE) o Expresiones de Tabla Común proporcionan una solución elegante para este problema, permitiendo crear conjuntos de resultados temporales que pueden ser referenciados dentro de una consulta principal.

Una CTE es esencialmente una tabla temporal con nombre que existe únicamente durante la ejecución de una consulta específica. Actúa como una subconsulta con nombre que puede ser referenciada múltiples veces dentro de la misma instrucción SQL, mejorando significativamente la legibilidad y modularidad del código.

```sql
-- CTE Básico
WITH mi_cte AS (
    SELECT columna1, columna2 FROM tabla1
    WHERE condicion = 'valor'
)
SELECT * FROM mi_cte;
```

Legibilidad Mejorada: Las CTE permiten descomponer consultas complejas en partes más pequeñas y comprensibles, similar a como las funciones simplifican el código en programación.

Reutilización: Una CTE puede ser referenciada múltiples veces dentro de la misma consulta, evitando la duplicación de lógica compleja.

Alternativa a Subconsultas: Proporcionan una alternativa más limpia y legible a las subconsultas anidadas, especialmente cuando se necesitan múltiples niveles de agregación.

Base para Recursividad: Las CTE recursivas permiten trabajar con estructuras jerárquicas y realizar operaciones que serían extremadamente complejas con SQL tradicional.

REFERENCIAS BIBLIOGRÁFICAS

1. ANSI. (1986). SQL-86 Standard. American National Standards Institute.

2. ANSI. (1989). SQL-89 Standard. American National Standards Institute.

3. ANSI. (1992). SQL-92 Standard. American National Standards Institute.

4. ANSI. (2003). SQL-2003 Standard. American National Standards Institute.

5. ANSI. (2006). SQL-2006 Standard. American National Standards Institute.

6. Date, C. J. (2004). An Introduction to Database Systems (8th ed.). Addison-Wesley.

7. Draw.io. (2023). Draw.io User Guide. Draw.io.

8. Elmasri, R., & Navathe, S. B. (2011). Fundamentals of Database Systems (6th ed.). Pearson.

9. ISO. (1987). Information technology - Database languages - SQL. International Organization for Standardization.

10. MySQL. (2023). MySQL 8.0 Reference Manual. Oracle Corporation.

11. Oracle. (2023). Oracle Database SQL Language Reference. Oracle Corporation.

12. PostgreSQL. (2023). PostgreSQL 13.0 Documentation. PostgreSQL Global Development Group.

13. Ramakrishnan, R., & Gehrke, J. (2003). Database Management Systems (3rd ed.). McGraw-Hill.

14. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2010). Database System Concepts (6th ed.). McGraw-Hill.

15. SQL Server. (2023). SQL Server 2019 (15.x) - SQL Server 2019. Microsoft Corporation.

16. Ullman, J. D., & Widom, J. (2008). A First Course in Database Systems (3rd ed.). Pearson.

17. W3Schools. (2023). SQL Tutorial. W3Schools.

18. W3Schools. (2023). MySQL Data Types. W3Schools.

19. W3Schools. (2023). MySQL Functions. W3Schools.

20. W3Schools. (2023). SQL Server Data Types. W3Schools.

21. Grafiati. (2023, October 13). Literatura académica sobre el tema "MySQL". Recuperado el 5 de noviembre de 2023, de https://www.grafiati.com/es/literature-selections/mysql/

22. BIBA. Biblioteca de la Universidad de la Rioja. (s.f.). Bases de datos - BIBA. Bibliografía básica. Recuperado el 4 de noviembre de 2023, de https://biblioteca.unirioja.es/biba/pag3.php?-cod=824

23. Universidad Complutense de Madrid. (s.f.). Bibliografía para las prácticas. Recuperado el 5 de noviembre de 2023, de http://www.fdi.ucm.es/profesor/fernan/DBD/bibliopr%E1cticas.pdf

24. Universidad Siglo 21. (s.f.). 3 - Libro - Bibliografia - 3 SQL Contenido 3 Introducción 104 3 Algo de historia del lenguaje SQL. Recuperado el 5 de noviembre de 2023, de https://www.studocu.com/es-ar/document/universidad-siglo-21/base-de-datos-i/3-libro-bibliografia/31992874

25. Scribd. (s.f.). BD Bibliografía | PDF | Servidor SQL de Microsoft. Recuperado el 5 de noviembre de 2023, de https://es.scribd.com/document/526240758/BD-Bibliografia

26. Grafiati. (s.f.). Bibliografías: "Structured Query Language (SQL)". Recuperado el 5 de noviembre de 2023, de https://www.grafiati.com/es/literature-selections/structured-query-language-sql/